
Data Structures

BS (CS) _Fall_2025

Lab_04 Manual



Learning Objectives:

1. Array based List

Array based List in C++

What is a List?

A List is a collection of items of the same type arranged in sequence. Basically, a flexible structure that can grow and shrink on demand. In a list element can be:

- Inserted
- Accessed
- Deleted

at any position!

Everyday examples:

- Grocery list
- To-do list
- List of assignments
- Recipe ingredients

Key Properties:

- Can be empty (no elements).
- Can have one or many elements.
- **Each element has a position/index:**
 - The first element is at position 1 (or 0 in C++).
 - The last element is at position n.
- **Elements are ordered linearly:**
 - $a_1, a_2, a_3, \dots, a_n$

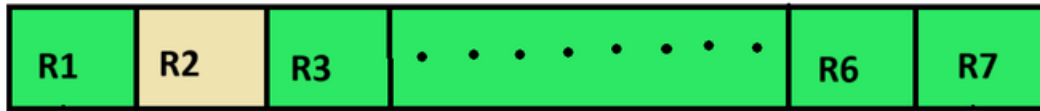
List as an Abstract Data Type (ADT)

When we talk about a List ADT, we describe what operations a list should support, not how it is implemented.

Basic Operations:

- **Create / MakeEmpty :-** Start with an empty list.
- **isEmpty() :-** Check if the list has no elements.
- **isFull() :-** Check if the list has reached max capacity.
- **size() :-** Get the current number of elements.
- **insert(x, p) :-** Insert element x at position p.
- **delete(p) :-** Remove element at position p.
- **retrieve(p) :-** Get the element at position p.
- **replace(p, x) :-** Replace element at position p with x.

- **locate(x)** :- Find the position of element x.
- **printList()** :- Display all elements.



Array-Based Implementation

We store the list inside a single array (contiguous memory).

We also keep track of:

- **Capacity**: maximum elements array can hold.
- **Size**: how many elements are currently in the list.

```
Capacity = 8
Size = 4
Array = [ 10, 20, 30, 40, _, _, _, _ ]
```

Main Operations

Access (Get / Set)

Access by index is instant because array indices are direct.

Example: `get(2)` :- returns 30.

```
Index:   0    1    2    3
Array: [10 | 20 | 30 | 40]
```

Insertion

- **At the End (Push Back)**
Just place the new element at the next empty slot.
Example: Insert 50

```
Before: [10, 20, 30, 40, _, _, _, _]   Size=4
```

```
After : [10, 20, 30, 40, 50, _, _, _] Size=5
```

- **In the Middle**

You must shift elements to make space.

Example: Insert 25 at index 2:

```
Before: [10, 20, 30, 40, 50, _, _, _] Size=5
Step 1: Shift [30,40,50] → one step right
        [10, 20, 30, 30, 40, 50, _, _]
Step 2: Place 25 at index 2
        [10, 20, 25, 30, 40, 50, _, _] Size=6
```

Deletion

To remove an element, shift left the remaining elements.

Example: Remove element at index 2 (25):

```
Before: [10, 20, 25, 30, 40, 50, _, _] Size=6
Step 1: Shift [30,40,50] → one step left
        [10, 20, 30, 40, 50, 50, _, _]
Step 2: Size = 5
Result: [10, 20, 30, 40, 50, _, _, _]
```

Searching

To find an element, scan the list linearly.

Example: Search 40:

```
[10, 20, 30, 40, 50]
Check each until found at index 3.
```

An array-based list is a list stored in a contiguous block of memory, giving fast access but slow insert/delete when not at the end.

Exception Handling in c++

What Is Exception Handling?

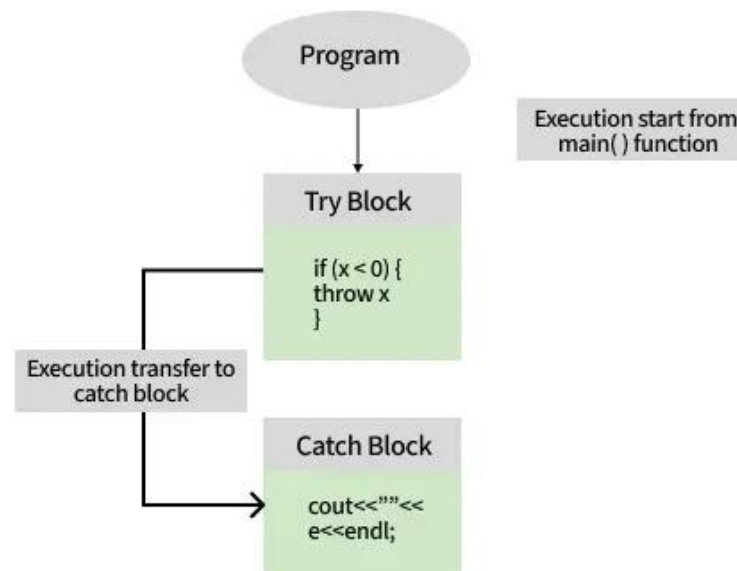
- **Definition:** A structured way to manage runtime errors, unexpected situations like division by zero, invalid memory access, or file operation failures.
- **Purpose:** Instead of letting a program crash abruptly, exception handling enables graceful recovery or controlled termination.
- Separates the normal program logic from the error-handling logic, improving code clarity and maintainability.

How It Works: throw, try, and catch

1. **throw:** Signals that an error has occurred; used to raise an exception.
2. **try:** Wraps code that could potentially throw an exception.
3. **catch:** Captures the thrown exception and defines how to handle it.

Flow:

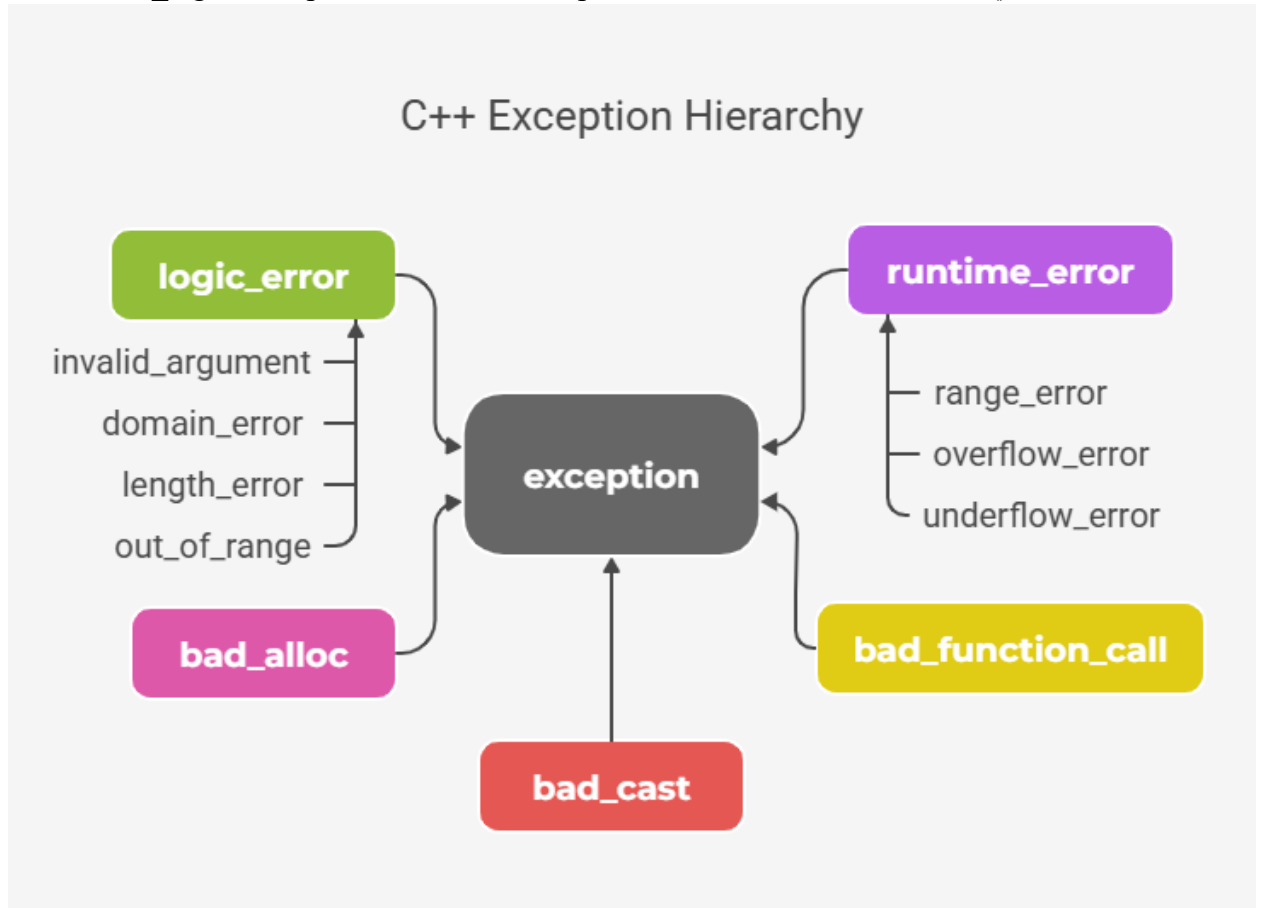
1. Code in try executes.
2. An error triggers throw, ending try abruptly.
3. Execution jumps to the first matching catch block.
4. Post-catch behavior allows safe continuation or termination.



Types of Exceptions to Throw

You can throw:

- Built-in types (e.g., `throw -1;`), simple but limited context.
- Standard exceptions (from `<stdexcept>`) like `std::out_of_range`, `std::overflow_error`, or `std::invalid_argument`- provide richer, descriptive error information via `what()`.



- Custom exceptions (user-defined classes), inherit from `std::exception` or its subclasses for tailored messages and better integration.

Syntax & Examples

Basic Try-Catch Structure:

```
try {  
    // risky operations  
    throw std::runtime_error("Something went wrong");  
}
```

```

catch (const std::runtime_error& e) {
    std::cerr << "Runtime error: " << e.what();
}
catch (const std::exception& e) {
    std::cerr << "Exception: " << e.what();
}

```

Throwing Built-In Type vs. Standard Exception:

```

// Built-in type-less informative
throw -1;

// Standard exception-more descriptive
throw std::out_of_range("Index out of range");

```

Best Practices (From GeeksforGeeks):

- Avoid using exceptions for normal control flow use only for unexpected, exceptional states.
- Catch by reference to const, e.g., `catch (const std::exception& e)`, to avoid object slicing and unnecessary copying.
- Prefer specific catch blocks before catching `std::exception` or using `catch(...)`. This allows tailored responses to different errors.

Applied Example for Your Lab (with Templates)

Context: Let's refine the `subList(start, end)` function to handle multiple error conditions using appropriate exceptions.

```

template <typename T>
List<T> List<T>::subList(int start, int end) {
    if (counter == 0) {
        throw std::underflow_error("List is empty – cannot extract a sublist");
    }
    if (start < 0 || end >= counter) {
        throw std::out_of_range("Start or end index is out-of-range");
    }
    if (start > end) {

```

```

        throw std::invalid_argument("Start index cannot be greater than end
index");
    }

    List<T> result(end - start + 1);
    for (int i = start; i <= end; ++i) {
        result.insert(arr[i]);
    }
    return result;
}

```

Usage with Exception Handling:

```

try {
    auto sub = lst.subList(3, 1); // Invalid: start > end
}
catch (const std::underflow_error& e) {
    std::cerr << "Empty list error: " << e.what() << std::endl;
}
catch (const std::out_of_range& e) {
    std::cerr << "Index error: " << e.what() << std::endl;
}
catch (const std::invalid_argument& e) {
    std::cerr << "Argument error: " << e.what() << std::endl;
}
catch (const std::exception& e) {
    std::cerr << "General error: " << e.what() << std::endl;
}

```

This structure demonstrates throwing different exceptions for different violations, and catching them with specific handling blocks, followed by a generic fallback.

Why This Matters in Your Task

- Your List<T> implementations must be robust and generic, using templates.
- Clear semantics: throwing relevant exceptions (underflow, overflow, out_of_range, invalid_argument) communicates intent and errors unambiguously.
- Proper handling with try-catch ensures errors are caught, reported, and don't cause crashes.
- To ensure that students follow industry-standard practices in error management.